

BACHELOR INFORMATICA



UNIVERSITY OF AMSTERDAM

Implementing Multi-Party Computation from Quantum Oblivious Transfers

D.R.B. van Wijk

June 19, 2026

Supervisor(s): Dr. Garazi Muguruza Lasa, Prof. dr. Paola Grosso

Signed: Signees

Abstract

Current techniques for MPC rely on oblivious transfer (OT), whose standard constructions are based on current public key infrastructure (PKI), making them vulnerable to attacks by quantum computers. In this thesis, we investigate whether quantum oblivious transfer (QOT) can improve on classical OT in existing secure multiparty computation (MPC) protocols. We implement a 1-out-of-2 QOT protocol in the SquidASM quantum-network simulator and extend it to the different oblivious-transfer variants required by MPC. Our construction combines quantum communication with classical bit commitment, error correction, and hashing. Benchmarking under current and optimistic hardware assumptions shows that low-density parity-check (LDPC) codes outperform Reed–Solomon codes at lower leakage rates, and that Carter–Wegman hashing scales more efficiently in the number of bits than Toeplitz hashing. The classical post-processing, however, contributes only a small fraction of the total runtime, since the dominant cost is generating and measuring sufficiently many high-fidelity entangled pairs. For a realistic level of security, the protocol requires on the order of millions of entangled states, leading to execution times of weeks per oblivious transfer even under optimistic entanglement-generation rates. Although all OT variants needed for OT-based MPC can be constructed from our 1-2 QOT implementation, the current performance gap means that QOT is, for now, mainly a proof of concept rather than a practical alternative in deployed MPC systems.

Contents

1	Introduction	6
1.1	Research question	6
1.2	Thesis Outline	7
2	Theoretical background	8
2.1	Multiparty Computation	9
2.2	Oblivious Transfer	9
2.3	Quantum Cryptography	10
2.3.1	Quantum communication	11
3	Related Work	12
3.1	Theoretical QOT	12
3.2	OT for MPC	12
3.3	Practical QOT	13
4	Simulation Environment	14
4.1	Using SquidASM	15
4.1.1	Creating an application	15
4.1.2	Classical Communication	16
4.1.3	Quantum communication	16
4.2	Simulator configuration	17
4.3	Simulation setup	17
5	Implementing 1-2 QOT	19
5.1	Implementation	19
5.2	Benchmarking 1-2 QOT	24
5.2.1	Error rates	24
5.2.2	Leakage rates	25
5.2.3	Hash times	26
5.2.4	Total times	27
5.3	Security Bound	29
6	OT-variants for MPC	31
6.1	1-2 bit OT	31
6.2	1-out-of-N OT	31
6.3	Rabin-string-OT	33
7	Discussion	35
7.1	Ethical Considerations	36
8	Conclusion	37
A	1-2 QOT Protocol	41

B	Simulating Parameters	43
B.1	Current	44
B.2	Optimistic	45

Introduction

As the use of digital applications keeps growing, so does the importance of secure data transfer. Secure multiparty computation (MPC) lets parties perform joint computations without having to reveal their private data. Some prominent techniques for MPC in federated settings, where participants might distrust the implementations of other participants, rely on the oblivious transfer (OT) cryptographic protocol. The problem is that this OT relies on current public key infrastructure (PKI), whose most widespread implementations are vulnerable to being broken by quantum computers. In addition, the handling of public keys can require non-trivial networking and security expertise, which not all organizations that would benefit from MPC have. A possible solution could be implementing these OT algorithms using quantum techniques, which promises security without utilizing PKI.

The aim of this thesis is to implement a quantum oblivious transfer (QOT) algorithm, based on a combination of techniques described in current literature. We do this by first implementing 1-out-of-2 OT in a quantum setting, then extending it in a way such that it can be used by classical MPC protocols. However, a downside of the current quantum hardware is that it is very noisy and prone to errors, since the qubits decohere rapidly over long distances. To test whether the protocol correctly accounts for this, these algorithms will be tested in a quantum simulator with parameters mimicking realistic circumstances. Performing well on these tests is critical to determine the viability of the algorithm in the context of secure multiparty computation.

1.1 Research question

The goal of this thesis is to lay the groundwork for combining a QOT implementation with a classical secure multiparty computation protocol, leading to the following research question:

“Can quantum oblivious transfer serve as an improvement for classical OT within existing multiparty computation protocols?”

In this thesis, “improvement” refers both to cryptographic assumptions and to concrete performance. On the one hand, QOT avoids the use of classical public-key infrastructure and is therefore attractive in a post-quantum setting. On the other hand, quantum communication is known to be slower and more resource-intensive than classical communication, which means that the improved security needs to be worth the trade-off in time taken.

Because of the broad scope of this subject, and the natural dependency between implementation and integration, this question will be split into two parts:

1: *“Is it possible to implement a quantum-enhanced Oblivious transfer protocol inside current entanglement based quantum-network simulation platforms?”*

This part will mainly focus on implementing the basic 1-out-of-2 OT in the NetSquid entanglement-based quantum network simulator, forming a proper basis for further development. This implementation will be tested and benchmarked at different noise levels to assess its robustness under realistic quantum conditions.

2: *“Is it possible to construct the QOT variants needed for a quantum-secure multiparty computation protocol?”*

In this stage, the OT variants necessary for the MPC protocol of Ishai, Sahai, and Prabhakaran [1] will be constructed and checked for their efficiency. This comes down to the extension to string-OT and Rabin-OT, this demonstrates that the implemented OTs are able to offer the same black box functionality with the security guarantees of quantum oblivious transfer.

1.2 Thesis Outline

The first part of this thesis lays out the necessary context. Starting with Chapter 2, which explains the background on this area of cryptography, as well as the quantum techniques used. Next, Chapter 3 describes the current state of Quantum Oblivious Transfer in the literature. The last piece of context is the environment in which the quantum code is written and simulated, as stated in Chapter 4. The implementation of the 1-out-of-2 QOT is demonstrated in Chapter 5 alongside sub-protocol choices made during developing, and its benchmarking results. Followed by Chapter 6, in which the other OT variations are constructed.

Lastly, the thesis ends with the discussions and conclusions in Chapter 7 and Chapter 8, recapping the gained insights and offering suggestions for future research.

Theoretical background

Cryptography is the act of writing a message in ciphertext, usually done by translating the plaintext according to some key, with the aim of protecting a secret [2]. Encrypting is translating the message to ciphertext, whereas decrypting is reversing it back to plaintext. When the same key is used for encrypting and decrypting, this is called symmetric encryption. The downside is that both parties previously need to have agreed upon this secret key, which can be a problem in the modern internet: if two parties have not yet ensured a secure means of communication, there is no way for them to agree on that key. Asymmetric encryption, also known as public-key cryptography, aims to solve this problem. Here, different keys are used for encryption and decryption, and one of them can be made public without compromising the other. In general, asymmetric encryption schemes are much more computationally intensive than symmetric ones, which means that in practice asymmetric encryption is used for the initial agreement of a key, after which symmetric encryption is used for the rest of the communication. This operating scheme is known as hybrid encryption.

The security of asymmetric encryption relies on the computational hardness of certain mathematical problems. For example, many widely used schemes, such as RSA, are built upon the integer factorization problem. Other schemes, such as elliptic curve cryptography, rely on the discrete logarithm problem instead. In both cases, these problems are considered computationally infeasible for classical computers to solve at the scale used in practice, making the encryption schemes secure enough that no realistic attacker could break them by brute force. This security is no longer guaranteed when considering the development of quantum computers. Peter Shor developed a quantum algorithm for finding the prime factors of an integer, commonly known as Shor's algorithm. This algorithm has evidence of a superpolynomial speedup compared to the best known classical algorithms [3]. Crucially, Shor's algorithm can also be adapted to solve the discrete logarithm problem, meaning that it threatens both families of asymmetric encryption schemes described above. This makes it much more feasible to break these encryption schemes, undermining the security assumptions they rely on.

Besides encryption, cryptography enables a range of other primitives for secure communication. One such primitive is the commitment scheme: a protocol in which one party commits to a value without revealing it, and can later open the commitment to prove the value was not changed. This captures two properties: hiding, where the committed value remains secret until opened, and binding, in which the committing party cannot change the value after committing. Commitment schemes serve as a building block for oblivious transfer (OT), a primitive in which a sender holds two messages and a receiver obtains exactly one of them, without the sender learning which one was chosen. OT is in turn complete for secure multi-party computation (MPC), which is a technique for multiple parties to jointly compute a function over their private inputs without revealing those inputs to each other. These relationships form a hierarchy of reductions: commitment schemes enable OT, and OT enables MPC.

Classically, constructing OT requires a Public Key Infrastructure (PKI), meaning it inherits the

same vulnerability to Shor’s algorithm as asymmetric encryption. Quantum protocols for OT, however, do not require PKI, making them a possible direction in the post-quantum world. The case of commitment schemes is found to be more complicated: while at first it was thought that quantum commitment schemes were unconditionally secure, they in fact still require a one-way function (OWF) as a computational assumption, so they do not achieve full information-theoretic security. The rest of the theory and Chapter 3 will go into this in more detail.

2.1 Multiparty Computation

The aim of secure multi-party computation (MPC) is to carry out a joint computation of a function among multiple parties in a secure manner. This means that the result should be computed without the parties revealing to each other anything about the data beyond what can be derived from said result, meaning only the minimum necessary information is disclosed [4]. This requirement for privacy is one of the most basic properties that a MPC algorithm aims to ensure. Another property is correctness, which means that the protocol functions as expected when all parties are being honest. The property of security is harder to prove than the property of correctness.

Beyond privacy and correctness, several other properties are relevant. Since these required security properties vary per use case, no single fixed list can capture what security means in general. Instead, security is defined through the ideal-world paradigm. This paradigm describes a thought experiment in which, in the “ideal world”, an externally trusted and incorruptible party assists with the computation. In this experiment, all parties send their input to this trusted party, which in turn computes the desired function and passes the result on to the other parties. The goal is to create an algorithm in the “real world” that comes close to this ideal scenario.

An adversary controls parties, trying to disrupt normal protocol execution. Parties under the influence of an adversary are called corrupted. Sometimes it will be stated that an MPC algorithm is secure under certain assumptions of adversarial behaviour. This can be categorized depending on how much they deviate from the protocol:

1. **Semi-honest adversaries** follow the protocol specification, but attempt to obtain information that should remain private. A protocol that is secure against semi-honest adversaries guarantees that there is no inadvertent data leakage. Protocols secure against these kinds of adversaries are also called passively secure.
2. **Malicious adversaries** can arbitrarily deviate from the protocol specification. Protocols secure against malicious adversaries are also called actively secure. Protecting against these kinds of adversaries guarantees high security.
3. **Covert adversaries** can break the protocol, but have a specific chance of being detected while doing so. Thus, protocols secure against covert adversaries allow them to cheat when nothing is detected. This model can be useful in scenarios where there are penalties for adversaries when they are detected.

It is possible for several of these adversary types to cooperate in their behaviour. MPC protocols are often roughly divided into two types, based on how many adversaries may cooperate with each other: protocols that guarantee security under the assumption that there are always more honest parties than corrupted parties (an honest majority), and protocols that make no assumption about the number of corrupted parties. The second type offers stronger guarantees to each participant, since a party does not have to trust any other party to be honest, which is particularly useful in scenarios where only two parties need to compute a function together. However, such protocols often require stronger cryptographic assumptions or are less efficient than in the case of an honest majority.

2.2 Oblivious Transfer

As introduced in Section 2, oblivious transfer (OT) is a cryptographic primitive that serves as a key building block for MPC protocols. The most commonly used variant, 1-out-of-2 OT (1-2 OT,

or base-OT), involves two parties. One of the parties (Alice) wants to send either message m_0 or m_1 to another party (Bob), who has determined which message he wants by setting a choice bit $b \in \{0, 1\}$. The goal is to transfer m_b without Alice learning what b is, and without Bob learning the contents of $m_{\bar{b}}$ [5]. Several variants and extensions of the OT primitive exist to fit different security models and use cases, but all the variants considered in this work can be constructed from base-OT.

These OT primitives are able to form the building blocks for more complex MPC protocols. For example, by representing a function as a Boolean circuit and applying OT at the different gates, each party only learns what the circuit's output reveals. A popular protocol that uses this approach is Yao's Garbled Circuit. It proceeds as follows:

1. Alice generates a circuit with different randomly generated input labels and uses these labels to encrypt the resulting values in the truth table.
2. She shuffles these encrypted values, producing the garbled circuit, and sends it to Bob together with her own input labels. As the input labels consist of random data, Bob does not know the actual value each label holds.
3. Bob requests his input labels from Alice via OT. Because of the oblivious transfer, Alice does not know Bob's input.
4. Bob uses Alice's and his own input labels to unlock the values in the garbled circuit, being only able to decrypt the matching correct row.

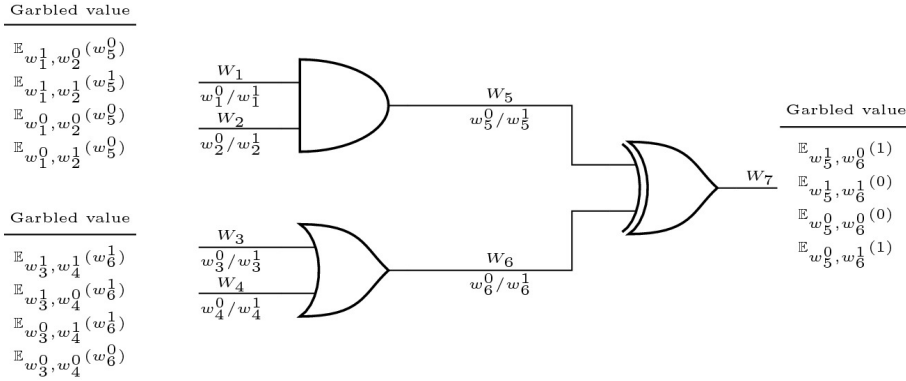


Figure 2.1: A garbled circuit depicting a situation with an AND gate and an OR gate, whose result is XOR'ed.

There are constructions of MPCs based on variants of Yao's garbled circuit technique that use OT as a black-box, meaning that the security properties of the underlying composable OT get lifted to the MPC. That is: if the OT is built only from one way functions, so is the MPC, without needing any other cryptographic assumption. Section 3.2 details the different OT variants and their use in MPC.

2.3 Quantum Cryptography

Quantum cryptography applies the principles of quantum physics to the design of cryptographic primitives [2]. Due to the laws of quantum mechanics, it is not possible to measure an arbitrary quantum system's state without disturbing it, making eavesdropping detectable by the legitimate parties. This property is widely used for quantum key distribution (QKD): assuming an authenticated classical channel and a quantum communication channel, a random key can be distributed between any two parties without requiring a previously shared secret. The resulting key is secure against an eavesdropper with unlimited computing power, making QKD a basis for information-theoretically secure key exchange.

2.3.1 Quantum communication

There are different ways of implementing the communication of qubits, but they can be divided into two main categories: prepare-and-measure protocols and entanglement-based protocols.

- **Prepare-and-measure protocols** rely on the transmission of individual quantum states from the sender to the receiver. The sender prepares each qubit in a randomly chosen basis (the computational basis $\{|0\rangle, |1\rangle\}$ or the Hadamard basis $\{|+\rangle, |-\rangle\}$) such that the receiver does not know which basis was used for each qubit. The receiver then measures each incoming qubit in a randomly chosen basis. After the exchange, both parties publicly compare which bases they used for each qubit. Only the results where both parties chose the same basis are retained, this process is known as sifting. The retained bits form the raw key. Most QKD today uses prepare-and-measure protocols, with BB84 being the most prominent example.
- **Entanglement-based protocols** make use of entangled photon pairs. Two photons are entangled when their measurement outcomes are correlated, even when spatially separated. This correlation only becomes apparent after the parties compare their results over a classical channel: no information is transmitted instantaneously between the photons. Any eavesdropping attempt requires an interception measurement, which disturbs the quantum state and introduces a statistically detectable increase in the error rate.

In this protocol, an entangled pair of qubits is prepared and one qubit is sent to each party. Both parties measure their qubit independently, each choosing a random basis. They then publicly compare bases and retain only the results where their basis choices matched, analogous to the sifting step in prepare-and-measure. The retained correlated outcomes can be used for key creation.

Because of the difficulty of achieving entanglement, the rapid degradation and the difficulty of storing qubits, prepare-and-measure protocols have been more widely deployed in practice. However, entanglement-based protocols have been gaining traction due to their stronger security properties, in particular their potential for device-independent security.

The next two chapters of this thesis will elaborate on the current state of Quantum Oblivious Transfer and our implementation choices for quantum-based networks.

Related Work

3.1 Theoretical QOT

The idea of using quantum communication for oblivious transfer was first proposed by Crépeau and Kilian [6]. They do this by making use of BB84 states, meaning that bits are encoded as either 0 or 1 using the computational or Hadamard basis. Measuring a qubit in the wrong basis irreversibly destroys the encoded information, enabling the obliviousness guarantee. Bennett, Brassard, Salvail, and Smolin [7] later presented a version of this protocol that is robust against transmission errors.

Unfortunately, these early protocols lacked rigorous security proofs. Later work attempted to prove their security by assuming that information-theoretically secure quantum bit commitment was possible, an assumption that was soon shown to be false by Lo [8] who presented impossibility results for quantum bit commitment. This no-go theorem left the security of QOT unclear and halted research for a while.

A breakthrough was achieved in 2009 when Damgaard, Fehr, Lunemann, Salvail, and Schaffner [9] showed that the protocol can be made simulation secure if the underlying bit commitments satisfy two additional properties: extractability and equivocality. With this, they obtained a provably simulation-secure QOT protocol under well-defined cryptographic assumptions.

A more recent result by Bartusek, Coladangelo, Khurana, and Ma [10] strengthens this: they show that the same type of security can be achieved from the weaker and more general assumption that quantum-hard one-way functions exist, using only black-box access to the underlying primitive. This is an important discovery, because one-way functions are widely regarded as the minimal assumption for essentially all of classical cryptography. Their result also implies that quantum-hard one-way functions suffice for general secure computation in the presence of quantum adversaries, in contrast to the classical setting, where such a black-box reduction from one-way functions is known to be impossible. Diamanti et al. [11] provide a noise-tolerant QOT protocol based on this structure. They are able to do so while avoiding bottlenecks affecting speed, and their version also takes ease of implementation into consideration.

3.2 OT for MPC

Several variants and extensions of the OT primitive exist to fit different security models and use cases. These variants can be used to build or strengthen MPC protocols, as demonstrated by the work of Ishai, Sahai, and Prabhakaran [1]. In their paper, they use these OT primitives to create a compiler that transfers MPC protocols with the condition of an honest majority, into protocols that remain secure even without an honest majority.

The most commonly used OT variants for MPC are:

1. **1-2 bit OT**: This is the most basic version of 1-2 OT. As described in Section 2.2, Alice

has two bits (m_0, m_1) , and Bob selects one of them via his choice bit $b \in \{0, 1\}$, without Alice knowing which one. This is the most limited version of its kind, as only a single bit is transmitted. It is, however, theoretically the easiest to construct and can be used as a basis to construct other OT primitives.

2. **1-2 string OT**: This subvariant is the one most commonly implemented as quantum oblivious transfer. This case is similar to 1-2 bit OT, but instead of two single bits, Alice is able to send two k -bit strings as messages to Bob. In most MPC protocols, this version of OT is used to transfer short cryptographic keys or wire labels rather than single bits. For example, garbled-circuit protocols (see Section 2.2) use one 1-2 string OT per input wire to deliver the correct wire label to the evaluator.
3. **1-out-of- N string OT**: This version is a generalisation of 1-2 string OT. Instead of choosing one message out of two possibilities, Bob is able to choose one out of an arbitrary number of messages (N) using an index i . This OT variant is useful when a party must select one item from a larger database, as in private information retrieval or certain oblivious array-access subroutines inside MPC protocols. However, from a complexity-theoretic perspective, 1-out-of- N OT does not give strictly more power than 1-2 OT. In particular, this means that it is possible to use 1-2 OT for the construction of 1-out-of- N OT, as shown by [12].
4. **Rabin string OT**: This was the first form of oblivious transfer, proposed by Michael Rabin [13]. In this variant, a sender sends a message that the receiver obtains with probability 50%, while the sender remains oblivious as to whether the transmission was successful. This randomness in Rabin OT can be useful in MPC. For example, Ishai, Sahai, and Prabhakaran [1] use Rabin OT for the creation of one-time pads for the actual messages to be transferred. In their paper, they realise this Rabin OT protocol by using 1-out-of- N OT with a constant communication overhead.

Through these reductions, it is possible to construct different OT variants from one another. In our case, we can transform 1-2 string OT into stronger primitives in the following manner: **1-2 string OT** $\rightarrow$ **1-out-of- N string OT** $\rightarrow$ **Rabin string OT**. Once we are able to construct these variants using QOT as a base primitive, we are one step closer to demonstrating the feasibility of an MPC protocol under the assumption of quantum oblivious transfer. These reductions and their potential within MPC will be discussed in more detail in Chapter 6.

3.3 Practical QOT

Most of the work described above is theoretical. On the practical side, quantum-network implementations of cryptographic protocols have been gaining traction as quantum hardware matures. A recently published article by Hanouz, Kaplan, Tournebize, Liao, and Marin [5] demonstrates that currently existing prepare-and-measure QKD hardware can be used for other quantum cryptography tasks, such as quantum oblivious transfer and quantum tokens. They do this by first implementing these algorithms in a simulator that replicates the output of real hardware, and afterwards also testing on actual quantum communication hardware to validate the results and measure performance. The QOT protocol implemented is from the preprint of Diamanti et al. [11]. For practical reasons, Hanouz et al. switch the non-standard ERE-based commitment scheme used by Diamanti et al. back to the original random-oracle-based construction from Bartusek, Coladangelo, Khurana, and Ma [10]. Mainly, this is because the properties that the ERE-based commitment scheme provides (equivocality and extractability) are needed for the simulation-based proofs of security, and are not needed for proving the functionality of the algorithms on the hardware. Instead, Hanouz et al. implement commitments using SHA-256 hash functions, which are idealized to random oracles in the security proofs.

The implementation in this thesis builds on a similar approach, using the SquidASM platform on top of the NetSquid simulator to implement and benchmark QOT under realistic noise conditions. Rather than targeting theoretical minimality of assumptions, the focus here is on whether QOT can be implemented robustly enough to serve as a drop-in replacement for classical OT in existing MPC protocols.

Simulation Environment

Our simulation environment is implemented using the SquidASM simulator for quantum internet applications. SquidASM can be used as an environment to develop quantum applications, as well as to test the effect of a network on these applications, such as error tolerance, throughput, and latency. This makes it useful for testing our quantum oblivious transfer algorithm implementations.

SquidASM requires the instructions for the quantum communication itself to be written in NetQASM: a low-level instruction set architecture designed to abstract quantum operations for network controllers, enabling applications to run on a quantum network [14]. NetQASM provides an SDK, enabling us to also use Python for developing this part of the quantum application.

SquidASM uses NetSquid as its backend simulation engine for quantum communication. NetSquid is a simulator built by QuTech in Delft [15]. It is based on discrete events and offers support for simulating multiple aspects of quantum networks and modular quantum computing systems, from the physical layer and its control plane up to the application layer. The simulator is also able to model the effects of time on the performance of these systems.

The dependencies between these tools can be seen in Figure 4.1. The rest of this chapter explains how we used this “simulation stack” to implement our algorithm and perform our tests. First, we show how SquidASM is used to create a quantum application. We then show how the simulator can be set up to model different networks. Lastly, we elaborate on the specific parameter values we used to simulate our quantum network. More information about the usage of SquidASM can be found in the documentation [16].

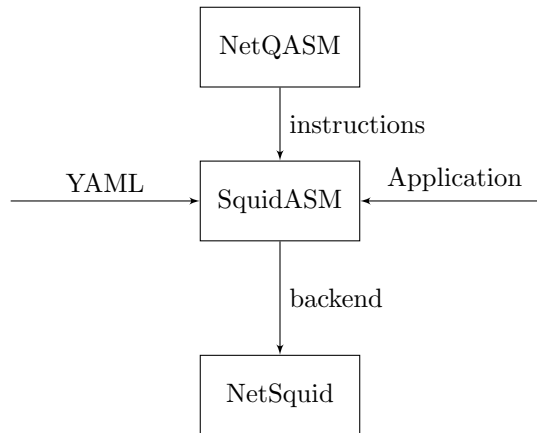


Figure 4.1: The used simulation stack.

4.1 Using SquidASM

Our setup consists of two hosts, each of which functions as a node in the network. These nodes can connect to other nodes using two different types of channels: classical and quantum. Classical channels allow regular communication between nodes, whereas quantum channels are used to share entangled qubits. The two hosts can directly communicate with each other using the classical channel. The quantum channels, however, are managed by a quantum network processing unit (QNPU), which is responsible for local qubit operations and EPR pair generation with the remote nodes. The link between the QNPU and the hosts is realized via the NetQASM connection. Figure 4.2 depicts this structure schematically.

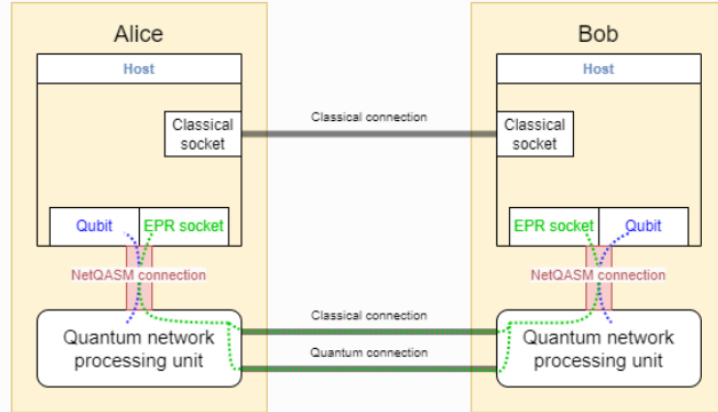


Figure 4.2: Schematic representation of the network setup using a NetQASM connection.

4.1.1 Creating an application

In SquidASM, a `Program` refers to the code being executed on one node. While an `Application` describes the collection of programs on multiple nodes, working together for a common goal. The applications reside in `application.py`, in which the programs are created by subclassing the main `Program` class, and implementing the required functions. It is common to write code for the different hosts in the same file. An example can be seen in Listing 4.1, where the setup of Alice and Bob's programs are shown.

Each `Program` requires the implementation of two functions: `meta()`, responsible for providing the additional data needed for communication, and `run()` which includes all the code to be ran for simulations. This includes the classic communication, quantum communication, and any post processing tasks.

```

1  class AliceProgram(Program):
2      PEER_NAME = "Bob"
3
4      @property
5      def meta(self) -> ProgramMeta:
6          return ProgramMeta(
7              name="program_name",
8              csockets=[self.PEER_NAME],
9              epr_sockets=[self.PEER_NAME],
10             max_qubits=2,
11         )
12
13     def run(self, context: ProgramContext):
14         # <-- The code contents of Alice's program here
15
16
17     class BobProgram(Program):
18         # <-- Contents of Bob's class here, as per the above

```

Listing 4.1: An Example of a Basic Program in SquidASM

4.1.2 Classical Communication

An example of classical communication between Alice and Bob can be seen in Listing 4.2. First, the `csocket` object is initialised for each communicating party by specifying the name of the peer they want to talk to, where `self.PEER_NAME` is the name of the peer node, as can be seen in lines 2 and 6. Line 3 demonstrates how Alice sends her message to Bob. Bob then receives the message using the Python `yield from` keyword, as seen in line 7, which suspends execution until a message is received.

```
1  ## Alice
2  csocket = context.csockets[self.PEER_NAME]
3  csocket.send(message)
4
5  ## Bob
6  csocket = context.csockets[self.PEER_NAME]
7  message = yield from csocket.recv()
```

Listing 4.2: Classical Communication Between two Hosts

4.1.3 Quantum communication

The workflow for quantum communication can be seen in Listing 4.3. The `epr_socket` is a socket for generating entangled qubit pairs on both nodes. It acts as a proxy for interacting with the QNPU: all instructions are first queued locally, then converted to NetQASM assembly and executed on the QNPU.

Interacting with the `epr_socket` is similar to the classical socket. As seen in lines 2 and 8, Alice registers a request for an EPR pair using `create_keep`, while Bob listens for the request using `recv_keep`. Since these functions can generate multiple pairs at once by setting `number=n` as an argument, they return an array. The first element is therefore unpacked using `[0]`. Once the pair is generated, gates can be applied to the qubit, such as the Hadamard gate in lines 3 and 9, before measuring it in lines 4 and 10.

To execute all queued instructions, `yield from connection.flush()` must be called, as seen in lines 5 and 11. It is important to note that `result` does not yet hold the measurement outcome after `result = qubit.measure()` is called in lines 4 and 10: the measurement is only performed once `flush()` is called. For this reason, `qubit.measure()` returns a future object, acting as a placeholder until the actual value is available. No logic or arithmetic operations should therefore be performed on this value before `flush()` is called. Uncommenting line 11 in Bob's code illustrates this: performing `result += 1` before flushing will cause an error.

```
1  ## Alice
2  epr_qubit = epr_socket.create_keep()[0]
3  epr_qubit.H()
4  result = epr_qubit.measure()
5  yield from connection.flush()
6
7  ## Bob
8  epr_qubit = epr_socket.recv_keep()[0]
9  epr_qubit.H()
10 result = epr_qubit.measure()
11 # result += 1 <-- Uncommenting will cause an error
12 yield from connection.flush()
```

Listing 4.3: Generating a Qubit-pair and Measuring it with the Hadamard Gate

4.2 Simulator configuration

To simulate a quantum network as accurately as possible, several configuration options are available and specified in a YAML file. It is possible to configure the individual nodes, as well as the links between them. This is done using three types of objects: stacks, links, and clinks.

- **Stacks** are the end-nodes and run the actual applications. Setting a name as well as the node type is required. It is also possible to provide additional settings for the model. The type can either be a generic qdevice or a nitrogen-vacancy (NV). For our use case, a generic qdevice is used.

Important parameters that we set are:

- **T1, T2:** The decoherence over time, modeled using **T1**, a energy or longitudinal relaxation time, and **T2**, a dephasing or transverse relaxation time.
- **single_qubit_gate_depolar_prob:** The chance that a random gate is applied to the one qubit involved in the operation.
- **two_qubit_gate_depolar_prob:** The chance that a random gate is applied to the two qubits involved in the operation.
- **Links** connect stacks and provide a way of generating EPR pairs between two nodes, requiring references to the two stacks they connect. It is possible to set a custom type, choosing between a perfect model, a depolarizing link, and a heralded link, with each type having its own configurable options. For our use case, a depolarizing link is used.

Important parameters that we set are:

- **fidelity:** How well entangled the successfully generated EPR pairs are.
- **prob_success:** The probability of success for a single entanglement attempt.
- **t_cycle:** The time in μs required for one cycle of preparing, sending, and measuring an entangled pair.
- **Clinks** are used for classical communication. Just like links, a clink also requires references to the two stacks it connects. It is possible to specify two kinds of clinks: an instant clink, where all communication is assumed to be without delay, or a default clink, with a configurable communication delay.

Using these properties, we can configure a model that matches our intended use case as realistically as possible. The specific simulation setup used, and how the parameters were obtained, will be discussed in more detail in Section 4.3.

4.3 Simulation setup

To gain an insight in the feasibility of QOT, it is necessary to look at realistic hardware scenarios for current and future quantum hardware, and see what kind of fidelity and performance is possible. We can use this as parameters when simulating the protocol.

A recent paper by Maiti, Avis, Kar, and Wehner [17] looks at the hardware requirements needed for quantum teleportation at an intercity wide scale, and perform simulations in the NetSquid platform: the same as we are using as the backbone for our simulations. For their experiments, the paper mentions two types of parameters: the baseline parameters, which represent the state-of-the-art experimental capability. And the optimistic parameters, which they think we'll be able to achieve in the near-term, indicated by experimental physicists at the Institute for Quantum Optics and Quantum Information, University of Innsbruck [18].

We can use the parameters to set the environment for the simulation. Some parameters can be used directly, some others are used to calculate the needed parameters. The used parameters can

be seen in table Table 4.1, and the final configuration files for the current and optimistic scenarios can be found in Appendix B.

Parameter $\lambda^{(i)}$	SquidASM Param.	Baseline $\underline{\lambda}^{(i)}$	Optimistic $\overline{\lambda}^{(i)}$
Quantum Device			
Coherence times	T1, T2	62 ms	4s
Single Qbit gate depolar prob.	single_qubit_gate_depolar_prob	0.02	0.004
Two Qbit gate depolar prob.	two_qubit_gate_depolar_prob	0.067	0.004
Link			
Prob. successful transfer (d=50km)	prob_success	5.95e-5	1.43e-3
Ent. fidelity f_m	fidelity	0.88	0.95
Cycle time	t_cycle	425 us	425 us

Table 4.1: Corresponding parameters with the paper to the ones used in the SquidASM simulator

Implementing 1-2 QOT

As written in Section 2.2, there are different versions of the oblivious transfer protocol. First, we have implemented the 1-2 OT variant, by building upon Protocol 1 from Hanouz et al. [5], which is based on Algorithm 5 from Diamanti et al.[11]. The difference being that Hanouz et al. replace the ERE-based commitment with the random oracle model used by Barutesk et al [10]. In addition, we tried two different error correcting and hashing schemes. These are benchmarked in Section 5.2 to determine which combination is the most promising for our base 1-2 Oblivious Transfer protocol.

This chapter will first explain our implementation of this protocol, along with the design decisions made. The second part of the chapter will benchmark the implementation under different circumstances to assess the feasibility of the algorithm and determine its optimal configuration.

The full flowchart for the algorithm can be found in Appendix A, and the Python code for the protocol and benchmarking can be found in the GitHub repository [19]. For the simulation of entangled states, the implementation from Section 4.1.3 is used, whereas the classical communication happens as described in Section 4.1.2.

5.1 Implementation

Step 1: State Distribution

First, Alice and Bob independently generate a random string specifying the basis θ they are going to measure the qubits in. Then, Alice creates a maximally entangled EPR pair, and measures her part with the pre-determined basis. Afterwards, she sends the other half to Bob, who in turn measures it with his basis choice. This gets repeated $2\lambda_{OT}$ times, with the resulting measurements depicted by x . λ_{OT} represents the amount of states that end up being used for the encryption, which means that this value influences the achievable security, as later described in Section 5.3.

Alice	Bob
<i>Input:</i> Messages (m_0, m_1)	<i>Input:</i> Bit $b \in \{0, 1\}$
<i>Common input:</i> $\lambda_{OT}, \lambda_{PQS}, h_i(), PRG(), Comm()$	
1.a Chooses $\theta_A \xleftarrow{\$} \{+, \times\}^{2\lambda_{OT}}$, and prepares EPR pair $(2 \text{ qubits } x_{AB}\rangle)$, of which she measures $ x_A^{\theta_A}\rangle$ and sends bob x_B .	1.b Measures his qubit with random basis $\hat{\theta} \xleftarrow{\$} \{+, \times\}^{2\lambda_{OT}}$ to obtain $\hat{x} \in \{0, 1\}^{2\lambda_{OT}}$.

Protocol 5.1: Alice and bob distribute the initial quantum states

With the shared measurements established, the next step is to ensure Bob has committed to his basis before Alice reveals hers.

Step 2: Bit Commitment

An important part of the quantum oblivious transfer scheme is the bit commitment. We want to make sure that Bob measures his received qubits immediately, instead of storing them until a later phase of the protocol. The security of the algorithm relies on Bob measuring his bits in a random basis that is independent of Alice's choice. After Bob measures, Alice sends her measurement basis so that Bob can construct his key using the positions where their bases match. If Bob is able to wait before measuring, he can lie about his chosen bases and reconstruct the keys for decrypting both messages. To prevent this, bit commitment is introduced into the protocol.

The commitment is done using SHA-256 hashing, with the procedure as follows:

1. For each bit and base b , Bob generates a random nonce $r \in \{0, 1\}^{\lambda_{HS}}$, and concatenates these bits and bases with their respective nonce. The result of this concatenation gets hashed, creating commitment t . This means that the calculation of commitment t can be written as $t = \text{SHA-256}(b||r)$. All values t are sent to Alice at once after calculation.
2. Then Alice randomly picks half of the indexes of the set to open for verification, depicted by $T \subset [2\lambda_{OT}]$ with $|T| = \lambda_{OT}$, and requests these values from Bob.
3. For these indexes, Bob opens the commitment by sending the values used to create it: b and r . This opening of the commitment is called d .
4. Alice verifies the values by checking if each commitment t matches its opening d for every index she specified in T . This can be written as function $\text{Verif}(t, d)$, outputting 1 for every matching commitment. Q_{tol} is the security parameter specifying the maximum percentage of error, resulting in the following check: $1 - \text{Verif}(t, d)/\lambda_{OT} \leq Q_{tol}$

Alice		Bob
2. Receives commitment t .	$\xleftarrow{\text{Classical}}$	2. Sends public commitment $t = \text{Comm}(\hat{x}) + \text{Comm}(\hat{\theta})$.
3. Chooses random subset $T \subset [2\lambda_{OT}]$ with $ T = \lambda_{OT}$ and sends T .	$\xrightarrow{\text{Classical}}$	3. Receives T .
4. Checks $1 - \text{Verif}(t, d)/\lambda_{OT} \leq Q_{tol}$, otherwise aborts.	$\xleftarrow{\text{Classical}}$	4. Opens and sends $d = \text{Open_comm}(\hat{x}_{ T}) +$ $\text{Open_Comm}(\hat{\theta}_{ T})$.

Protocol 5.2: Trough the Commitment scheme, Bob shows that he has measured the received qubits

With the commitment verified, Alice and Bob can proceed to reconcile their measurement bases.

Step 3: Basis Reconciliation

During this stage, Bob will provide the information so Alice can construct the rawkey without her ever knowing which message Bob has picked.

1. Alice sends her measurement basis θ to Bob.
2. Using θ , Bob constructs two sets containing all indexes not in T : I_b contains the indexes where Bob's and Alice's measurement bases match, and $I_{\bar{b}}$ contains all indexes where their bases do not match. Here, b is the index of the message Bob wants to read, meaning the matching set corresponds to his chosen message.

3. Alice receives I_0, I_1 not knowing which one contains the matching basis, and splits her measured values x into x_0, x_1 according to the received indexes, constructing the two different rawkeys used for encryption.

Alice		Bob
5. Sends θ_A .	$\xrightarrow{\text{Classical}}$	5. Receives θ_A .
6. Receives I_0 and I_1 .	$\xleftarrow{\text{Classical}}$	6. Computes $I_b = \{i \notin T : \theta_{A_i} = \hat{\theta}_i\}$ and $I_{\bar{b}} = \{i \notin T : \theta_{A_i} \neq \hat{\theta}_i\}$, sends I_0, I_1 .
7. Determines $\mathbf{x}_0 = \{x_i : i \in I_0\}$ and $\mathbf{x}_1 = \{x_i : i \in I_1\}$.		

Protocol 5.3: Bob determines the bits where their measurement-basis do and don't match for constructing the rawkey

With the two rawkeys constructed, error correction is applied to account for noise introduced during transmission.

Step 4: Error Correction

Depending on the exact setup of the quantum communication channel, there is a sizeable chance for a bit to flip during transmission. Due to this, Bob can end up with different values from Alice, even though they measured in the same basis. This is a problem, since the keys used by Alice and Bob need to be exactly the same to achieve a correct result. The ratio of incorrect (flipped) bits compared to the total bits is denoted by the Quantum Bit Error Rate (QBER).

To mitigate this, error correction is used. Alice generates error correcting codes, which Bob uses to fix the potentially flipped bits in his rawkey. A downside of this is that the transfer of these codes reveals information about the key. According to the Shannon's bound, one must transmit at least $nH(p)$ bits of extra information, where n is the number of bits in the rawkey, p is the probability of a bit flip, and H is the binary entropy function $H(x) = -x \log_2(x) - (1-x) \log_2(1-x)$. Choosing the syndrome length is a matter of tradeoffs: shortening the syndrome leaks fewer bits, but increases the chance of error correction failing.

For setting a maximum tolerated leakage, we introduce the leakage rate λ_ℓ , which is defined as:

$$\lambda_\ell = \frac{L}{n},$$

where L is the number of leaked bits and n is the length of the raw key. Intuitively, a higher QBER requires more parity information to successfully correct Bob's string, which increases L and therefore λ_ℓ . This is an important metric from a security point of view: any information revealed during error correction is also potentially available to an eavesdropper, impacting the security of the key. As stated previously by Shannon's bound, we need to expose at least $nH(p)$ bits of side information to offer enough error correction for a raw key with bit error rate p , so for each QBER p there exists a minimal leakage rate that any correct protocol must exceed. Estimating this minimal leakage rate as a function of the QBER tells us how much information we necessarily have to sacrifice to maintain correctness, and therefore where the efficiency–security trade-off lies.

As we see later in benchmark 5.2.1, the average QBER can be determined if a given fidelity of the link is known. Using this, the minimum required leakage rate can be estimated. Since the fidelity is not always known, or can change, it is better for Alice to calculate a QBER for the first half of the transferred bits. There doesn't need to be transferred any new data for this, as she knows those values from Bob after the bit commitment phase. Given that λ_{OT} is large enough and T is randomly chosen, this QBER can serve as an estimation for the other half of the bitstring.

Alice	Bob
8. Calculates QBER using Bob's exposed bitstrings.	
Generates and sends $\sigma_0 = \text{Synd}(\mathbf{x}_0)$, $\sigma_1 = \text{Synd}(\mathbf{x}_1)$,	$\xrightarrow{\text{Classical}}$ 8. Receives σ_0, σ_1 . Corrects $\hat{\mathbf{x}}_b = \text{Correct}(\hat{\mathbf{x}}_{ I_b}, \sigma_b)$.

Protocol 5.4: Alice constructs the syndrome used for error correcting Bob's rawkey

There are generally two strategies for detecting and correcting errors with as little redundant information as possible [20]. Firstly, it is possible to construct parity-check equations over subsets of bits. These equations are carefully designed, making it possible to detect and correct errors based on inconsistencies in the parity constraints. A classical example of this is the Hamming code, which enables error correction for single bits. Low-Density Parity-Check (LDPC) codes build upon this. Like Hamming codes, LDPC codes are defined through parity-check equations. They are different in the fact that they do not operate on small, tightly constrained blocks that yield exact algebraic corrections. Instead, LDPC codes operate on large blocks and rely on probabilistic iterative decoding. This technique allows for the correction of many more bits than the single bit gained with a Hamming code.

The second method is based on algebraic coding over finite fields, in which the data is interpreted as coefficients of a polynomial. The redundant data is created by evaluating the polynomial at extra points. Using these redundant points, the original polynomial can be reconstructed. The Reed-Solomon codes are a well known example of this method.

For their implementation of the oblivious transfer, Hanouz, Kaplan, Tournebize, Liao, and Marin [5] make use of one LDPC based algorithm. We have opted to look at both popular techniques and compare them to see which one would be preferred for the QOT, meaning that both the LDPC and RS algorithms are implemented.

Low-Density Parity-Check Codes (LDPC) are widely used for error correction in the transfer of data, for example in Wi-Fi and 5G, as well as in the storage of data on HDDs and SSDs.

An advantage of the LDPC correction, is that the amount of data leaked can be specified by setting the amount of rows in the parity check matrix.

For our implementation, we use the `ldpc` python [21] decoder with the following implementation:

1. First Alice computes syndrome using $\sigma(x) = Hx$ where x is her rawkey, and H a parity check matrix.
2. She sends this syndrome σ to Bob, who calculates his own syndrome using $\hat{\sigma}(\hat{x}) = H\hat{x}$
3. Bob then creates an error correction syndrome by XORing the values: $\bar{\sigma} = \sigma \oplus \hat{\sigma}$.
4. This error syndrome gets passed to the decoder in an attempt to get back to the original syndrome: $\sigma_{guess} = \text{ldpc_decoder}(H, r_e, \bar{\sigma})$ where r_e is the estimated (QBER) error rate.
5. Bob then tries to correct the original message by again XORing his bits with the new syndrome: $x_{corrected} = \hat{x} \oplus \sigma_{guess}$.

Our final implementation is a simplified version of Hanouz, Kaplan, Tournebize, Liao, and Marin [5]: they retrieve the parity check matrix from the LDPC4QKD [22] library which have different codes based on the QBER, and then post process it with a rate adaption matrix, based on the message length. As their implementation is optimised for the larger amounts of states used in QKD noise correction, we instead opted for using a random binary matrix as parity check matrix. Our implementation needs less states for correcting, which means smaller running-times, while still providing a solid foundation for understanding how LDPC codes work.

Reed-Solomon (RS): The RS error correction is widely used for correcting errors when storing data, for example in ECC-memory and QR codes. For our project, we use the RS encoder and

decoder provided by the `unireedsolomon` python library [23]. The encoder takes a block of data and adds extra error correcting bits to the end of it, enabling the recovery of the original data. A Reed-Solomon code is specified by $RS(n, k)$, where n is the total block length and k is the message length, meaning that the total number of added bits is represented by $t = n - k$. RS is able to detect any combination of up to t symbol errors, and correct up to $\lfloor t/2 \rfloor$ errors. The algorithm works due to the use of Galois fields, which causes the efficiency of the error correction to drop significantly at larger block sizes, meaning that in our case larger rawkeys need to be split up.

Because the error correction happens by appending the parity to the original data, the algorithm can be tuned to expose the minimum amount of symbols. This symbol size is specified in bytes, which makes the algorithm very efficient at correcting burst errors, as multiple errors within one byte will only count as a single symbol error. However, our rawkey is based on individual bits. Since the current noise models assume bit flips are not correlated to each other, the efficiency of burst correction will not be applicable in this case. Moreover, if every symbol contains 8 independently flipped bits, the chance of a symbol being corrupted grows by a factor of 8, making the error correction potentially very inefficient.

To mitigate this, we pack every bit into its own symbol. The downside of this solution is that it increases the size of the input for the RS algorithm, and the amount of rawkey that needs to be exposed for correcting errors. Since the algorithm only corrects $\lfloor t/2 \rfloor$ errors, we can only correct $\frac{n/8}{2} = n/16$ bit errors. In the worst case, where the leakage rate equals 1, this implies that we can correct a QBER of up to 6.25%.

Step 5: Encryption

In this last stage, Alice will encrypt the messages using a 2-universal hash of the rawkey and the nonce, a procedure also known as privacy amplification.

1. Alice generates two random bitstrings s_0, s_1 with length $n_0 + \lambda_{PQS}$, known as the nonce, and sends them to Bob.
2. Alice uses the 2-universal hash function to calculate a hash h using rawkey x_i and nonce s_i as input. First, the hash $h_i(x_i, s_i)$ is computed. Then, a pseudo-random generator (PRG) practically implemented by **SHAKE-256** is used to extend the hash to the required output length. Finally, the resulting value is XOR'd with the message to produce the encrypted message: $E_i = PRG(h_i(x_i, s_i)) \oplus m_i$. Alice sends both encrypted messages to Bob.
3. Using his reconstructed rawkey, the nonce, and the received messages, Bob decrypts his chosen message m_b using $PRG(h_b(\hat{x}_b, s_b)) \oplus E_b$.

Alice		Bob
9. Generates and sends seeds $s_0, s_1 \xleftarrow{\$} \{0, 1\}^{n_0 + \lambda_{PQS}}$. Constructs and sends $E_0 = PRG(h_0(x_0, s_0)) \oplus m_0$, $E_1 = PRG(h_1(x_1, s_1)) \oplus m_1$.	$\xrightarrow{\text{Classical}}$	9. Receives s_0, s_1, E_0, E_1 Decrypts $m_b = PRG(h_b(\hat{x}_b, s_b)) \oplus E_b$.

Protocol 5.5: Alice encrypts the message using her rawkey and the nonce, bob decrypts this using his error-corrected rawkey

For the hashing, two different 2-universal hashes have been tested: the Toeplitz hash algorithm used by Hanouz, Kaplan, Tournebize, Liao, and Marin [5], and the Carter-Wegman hashing algorithm.

Toeplitz Hashing: The formula used for Toeplitz hashing can be seen in Equation 5.1.

$$h_{T_a}(x) = T_a \cdot x \mod 2 \quad (5.1)$$

where T_a the nonce converted to a Toeplitz matrix
 x the input bitstring converted to a vector

Carter-Wegman Hashing: Our implementation follows the proposal from Carter and Wegman, shown in Equation 5.2. To obtain all input values, the bitstrings are converted into integers. The final result is again converted into a bitstring for later operations. The condition $P \geq U$ requires that the prime p is at least as large as the universe size U , i.e. the number of possible input values, ensuring the hash family is 2-universal.

$$\text{Considering } P \geq U: \quad h_{a,b}(x) = ((ax + b) \bmod p) \bmod n \quad (5.2)$$

where a, b are both halves of the nonce bitstring, split in half
 x the input bitstring message to be hashed
 p is the next prime greater than λ_{PQS}^2
 n the desired output length in bits, calculated as $2^{\lambda_{PQS}} - 1$

5.2 Benchmarking 1-2 QOT

To determine the feasibility of our implementation of the QOT algorithm, we have benchmarked individual parts as well as the whole protocol.

5.2.1 Error rates

First, we benchmarked the model to verify the dependence between the fidelity and the resulting Qubit Error Rate (QBER). The results are gathered by running the protocol up to step 3, where Alice can use the values Bob sends when opening the commitment to calculate the QBER. For different fidelities, the protocol has been executed 200 times to get a realistic estimate of this QBER. The parameter λ_{OT} has been set to 256. Error bars are also calculated and drawn, but they are not noticeable due to the low variance over this many runs.

Increasing the fidelity means decreasing the chance of a bit flipping, which should result in a lower QBER. As expected, we can see in Figure 5.1 that the QBER decreases approximately linearly as the fidelity of the link increases, meaning that as the link gets more stable, the number of flipped bits decreases.

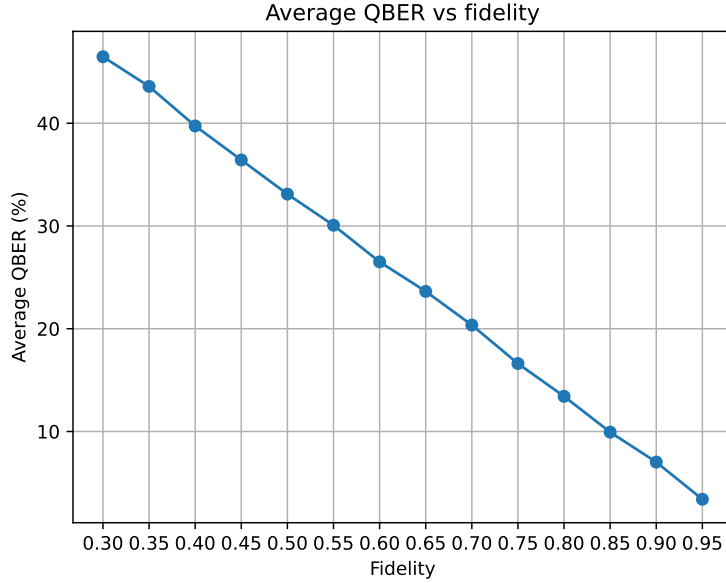


Figure 5.1: Measured QBER as a function of the configured link fidelity. Each point is averaged over 200 protocol runs. Error bars are smaller than the marker size.

5.2.2 Leakage rates

In these benchmarks we empirically determine the relation between the leakage rate of the error-correcting algorithms and properties of the quantum communication channel.

First we calculate the leakage rate for the two different error-correcting algorithms under the optimistic and current scenarios, as specified in Section 4.3. We gather the measurements by running 100 experiments per scenario for each leakage rate, and then looking at the percentage of successful decoding of the chosen message using the corrected rawkey. Figure 5.2 shows the results for the Reed-Solomon algorithm: it can be seen that for the current scenario, even with a very high leakage rate, the success rate is very low. The success rate in the optimistic scenario increases more steeply, but still requires a high leakage rate to reach a success rate stable enough to be usable. Meanwhile the LDPC error correction in Figure 5.2 shows better performance: the current and optimistic scenarios are much closer to each other in performance, and both perform better than the RS error correction algorithm.

These plots demonstrate that, although Reed-Solomon codes can in principle be tuned to leak only a limited number of bits, in our case they require a very high leakage rate before the protocol becomes reliable. LDPC codes achieve comparable or better success rates at lower leakage, making them the more practical choice for the rest of this work.

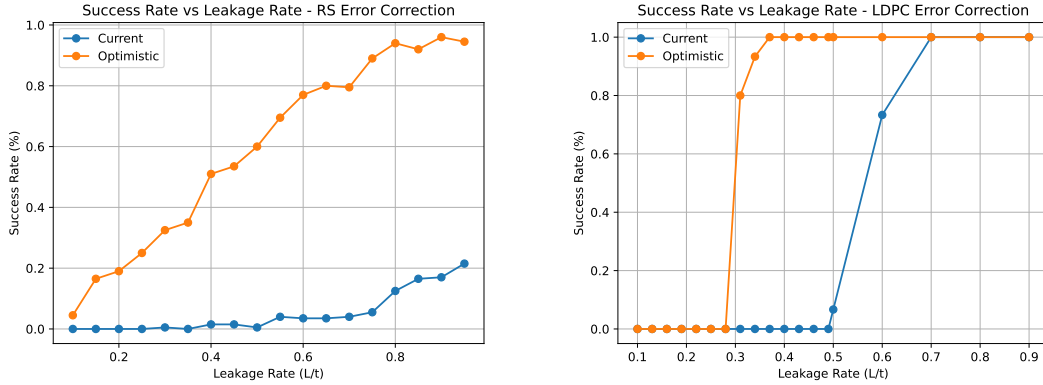


Figure 5.2: Comparison between success rates for the current and optimistic scenarios as a function of the leakage rate. For each data point the parameter. Showing RS (left) with $\lambda_{OT} = 256$, and LDPC error correction (right) with $\lambda_{OT} = 7680$

For application purposes, it is of interest to know the parameter that needs to be set to achieve a specific result. In the next benchmark we empirically determine, for a continuum of set fidelity, the minimal leakage rate at which the protocol still achieves a sufficiently high decoding success probability. In this case, we used LDPC error correction, as it yielded the best results in the previous benchmark.

We attained the results by executing the protocol 25 times for different steps of fidelity, with the leakage rate initially set to a lower bound based on previous results. The success rate is measured for these executions. If the success rate is below the threshold, the leakage rate λ_ℓ is gradually increased, up until the target threshold p_{success} is reached. The set leakage rate that passes this threshold gets noted as the minimum leakage rate. Thus, the resulting Figure 5.3 gives, for each fidelity, an estimate of the minimal leakage rate required to meet a specific desired reliability.

Interestingly, the RS error correction improves linearly as the leakage rate increases. This is in contrast to the LDPC correction, where there seems to be a turnaround point for the performance of the algorithm at a leakage rate of around 0.3. Overall, the LDPC error correction performs much better in our use case than the RS error correction.

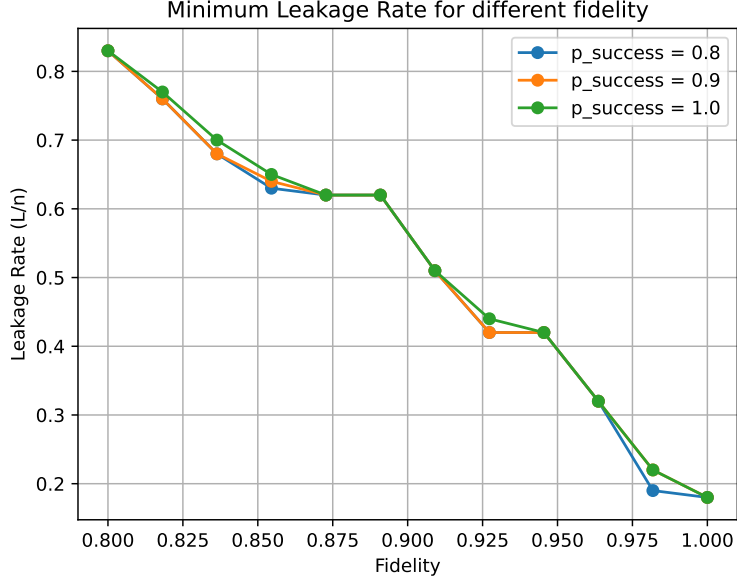


Figure 5.3: Estimated minimal leakage rate λ_ℓ required to reach a target decoding success probability p_{success} as a function of the link fidelity, using LDPC error correction. For each point, $\lambda_{OT} = 6917$ and the estimate is based on 25 runs per fidelity value.

5.2.3 Hash times

To determine the performance of the hashing algorithms, we benchmarked the time taken for the hashing algorithms to hash a randomly generated bitstring of a specific length. Every value is tried 100 times, and the average time is taken. The algorithms' seed size is set to $1.5 \times \text{message_length}$.

Figure 5.4 shows the time taken to hash the messages as their length increases. The Toeplitz hash seems to grow approximately linearly with the input length, while the Carter-Wegman hash increases much more slowly. This suggests that, for longer messages, Carter-Wegman hashing provides a significant performance advantage.

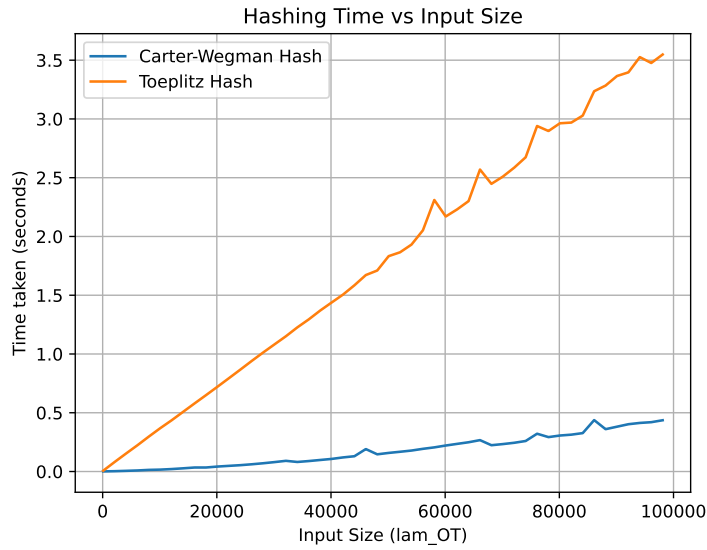


Figure 5.4: Measured time taken to hash a random bitstring with a given length for the two different hashes. The nonce length for the hashes was set to $0.5 \lambda_{OT} + \lambda_{OT}$, that is $1.5 \lambda_{OT}$.

5.2.4 Total times

To gain more insight into the performance bottlenecks of the algorithm, we measured the time taken for both the quantum communication and the classical post-processing in the protocol. This includes failed attempts as well, since they should not differ in the time required per run. The parameter λ_{OT} is varied to study the impact of the message length on the total protocol time. The quantum transfer time is obtained from the simulator, while wall-clock time is used for the post-processing.

Figure 5.5 shows that the quantum communication accounts for a considerable fraction of the runtime in the OT protocol, whereas the post-processing time is almost negligible, ranging from 0.001% to 0.0016%. There is also not much difference between the post processing time of the current and the optimistic quantum hardware. These findings are in stark contrast to those of Hanouz et al. [5], where post-processing takes up a little less than half of the total runtime. A possible explanation is the better performance of prepare-and-measure hardware compared to the entanglement-based variant. The plot also shows that the quantum transfer performs better in the optimistic scenario, although this has little impact on the post-processing time.

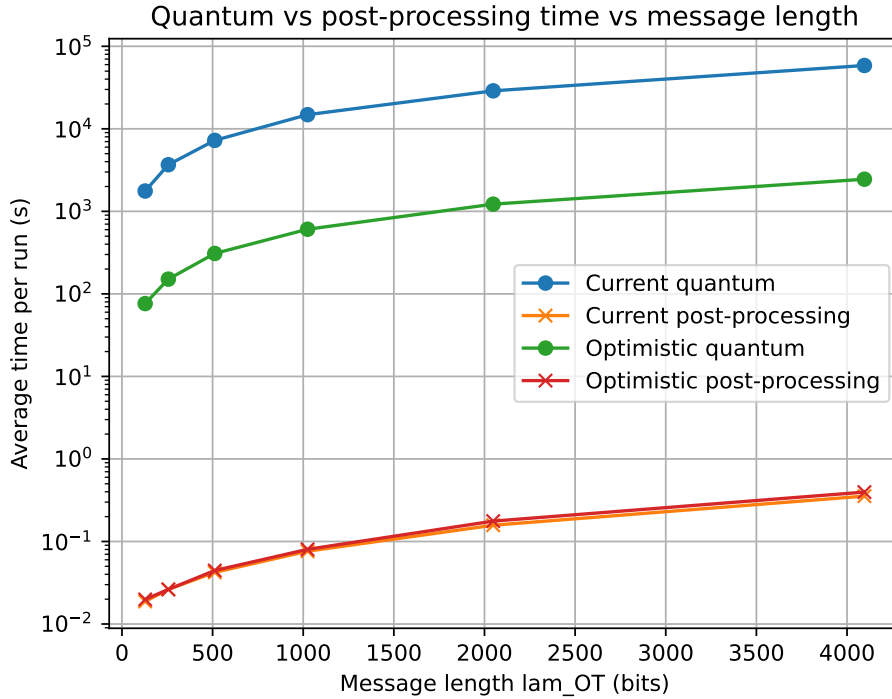


Figure 5.5: Time taken from the start of the protocol until the attempt at decoding the chosen message, for different values of λ_{OT} and for different stages. Using the Carter–Wegman hash and LDPC error correction.

To evaluate how useful the protocol is in a practical scenario, it is necessary to know the time required to achieve a successful transfer. The protocol is run repeatedly, and the time taken for each attempt is accumulated until a successful attempt is reached. This procedure is repeated 20 times for each parameter setting, and the results are averaged; error bars indicate the variance across runs. The maximum number of attempts is capped at 1000 to keep the total runtime manageable.

Figure 5.6 shows the average time required to achieve a successful OT transfer as a function of λ_{OT} for different leakage rates. The curves increase approximately linearly with λ_{OT} . A leakage rate of 0.3 performs consistently worse than leakage rates of 0.35 and 0.4, which can be explained

by Figure 5.2, where the success rate exhibits a clear turning point in the number of messages that the LDPC error-correction algorithm can reliably correct. This behavior implies that, in the regime where the error-correction algorithm consistently corrects all errors, the time required for a successful OT would be around one hour for $\lambda_{OT} = 6000$, corresponding to about 200 EPR pairs being transferred per minute.

We can compare these results to the OT performance estimated by Hanouz et al. [5], as shown in Figure 5.7. From this figure we read that 3.28×10^7 photons are transmitted in 12 minutes, which corresponds to 2.73×10^6 states per minute, so their hardware is roughly 1.4×10^4 times as efficient at generating a single OT. From this we can conclude that entanglement-based hardware is significantly slower for implementing OT than the prepare-and-measure hardware used by Hanouz et al.

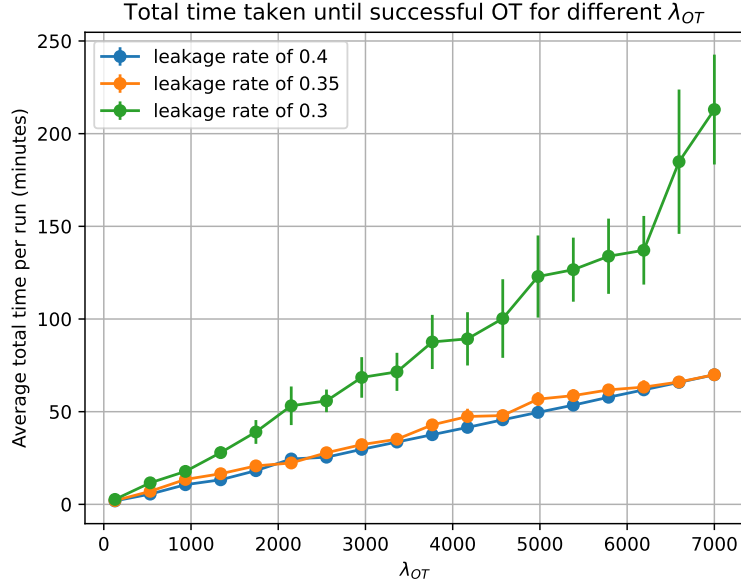


Figure 5.6: Average time needed to obtain a successful 1-2 QOT execution as a function of λ_{OT} for different leakage rates λ_ℓ . Each point is averaged over 20 runs; error bars indicate the variance between runs. The maximum number of attempts is set to 1000.

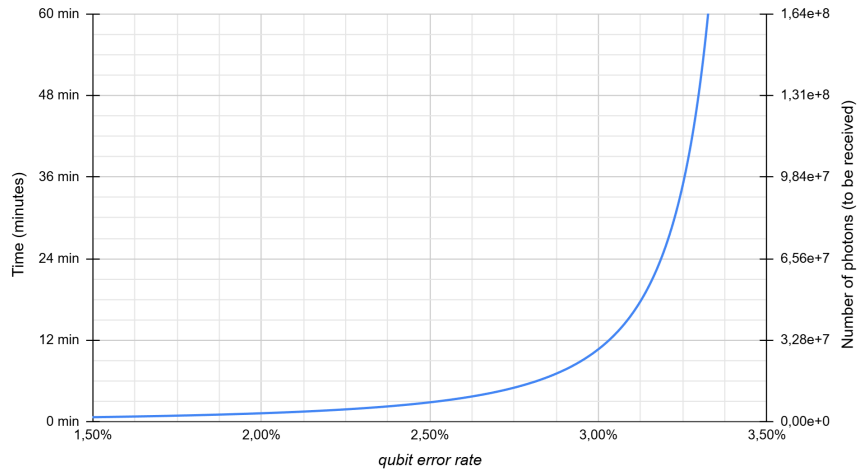


Figure 5.7: Estimation of the number of qubits to be received, and corresponding time required on Qline, to perform one OT as a function of the qber. The inefficiency of the error correction is set to 1.25. [5]

5.3 Security Bound

Just as described in Section 2.1 for MPC, the security of our QOT protocol is defined by the (trace) distance between the real world and the ideal world. In our setting, the real state is the joint state $\rho_{K_{\bar{b}}K_bE}$ of Alice's two messages K_0, K_1 (where $K_{\bar{b}}$ should remain hidden and K_b may be learned by Bob) and Bob's quantum side information E , while the ideal state is $\frac{1}{2^l}I \otimes \rho_{K_b} \otimes \rho_E$, where $K_{\bar{b}}$ is replaced by a uniform and independent l -bit string. The trace distance between these two states upper-bounds Bob's advantage between the real protocol and an ideal functionality, and thus quantifies how much information about $K_{\bar{b}}$ can leak in the worst case.

Hanouz et al. [5] specify their security bound based on a bound given by Diamanti et al. [11], and use it to derive a function that computes the required λ_{OT} for a given channel, leakage model, and target security level. In the original bound, additional parameters such as extra leakage terms are present. As in Hanouz et al., we set these to zero in our analysis. This leads to the following bound:

$$\Delta(\rho_{K_{\bar{b}}K_bE}, \frac{1}{2^l}I \otimes \rho_{K_b} \otimes \rho_E) \leq \frac{1}{2} \times 2^{-\frac{1}{2}[(\frac{1}{2}-\xi-h(Q_{\text{tol}}+\delta))\frac{\lambda_{OT}}{2}-q-l]} + \sqrt{6} \exp(-\frac{\delta^2}{100}\lambda_{OT}) + 2 \exp(-\frac{\xi^2}{2}\lambda_{OT}). \quad (5.3)$$

Where:

- Δ is the trace distance between the real and ideal states;
- ξ and δ are statistical fluctuation margins capturing the possible deviations between the observed sample statistics and the underlying population. The bound holds for any strictly positive values of ξ and δ for which $h(Q_{\text{tol}} + \delta)$ is well defined. Operationally, ξ and δ can be chosen to balance a larger key rate against tighter finite-size concentration bounds; they are therefore selected so as to minimize the final security parameter ϵ .
- Q_{tol} is the tolerable qubit error rate (QBER) for the protocol;
- $2\lambda_{OT}$ is the total number of received photons (or, equivalently, raw measurement outcomes);
- l is the output length of the 2-universal hash function used in the privacy amplification step of the post-processing (step 5 in our implementation). In our experiments we fix $l = 256$;
- q is the syndrome length of the error-correcting code used in step 4 of the post-processing. If n the raw key length, then $L = q$ and therefore $\lambda_{\ell} = L/n = q/n$.

Using (5.3), we perform a discrete grid search over ξ and δ to find, for each fixed $(Q_{\text{tol}}, \lambda_{\ell})$, the smallest value of λ_{OT} that still satisfies a target security level. For every calculated point of QBER we perform the following loop:

1. Fix Q_{tol} (using the QBER), the leakage rate λ_{ℓ} (one of the set to one of the three lines), and start with a low value of λ_{OT} .
2. Evaluate the right-hand side of (5.3) over a discrete grid of ξ and δ values
3. Take the minimum value of the bound over this grid as our approximation of the optimal choice of (ξ, δ) for that λ_{OT} .
4. Increase λ_{OT} until this minimum drops below the target overall security parameter ϵ_{sec} .
5. This is the minimum value of λ_{OT} , try this for the other leakage rates and start the loop again with a higher Q_{tol} .

The resulting minimal λ_{OT} is then plotted as a function of the QBER. For each QBER value we repeat this procedure for several leakage rates λ_{ℓ} to study how the amount of leaked information

affects the number of quantum states that must be transmitted. Following Hanouz et al., we set the overall security to $\epsilon_{\text{sec}} = 2^{-22}$, which is slightly above the typical “cryptographic” level of 10^{-10} but sufficient for comparing parameter regimes and illustrating the scaling behaviour. Choosing a smaller ϵ_{sec} would shift all curves upwards, i.e. increase the required λ_{OT} , but would not change their dependence on QBER and leakage.

Figure 5.8 shows the minimal λ_{OT} as a function of the QBER for several fixed leakage rates. We see that both the leakage rate and the QBER have a significant influence on the required λ_{OT} : for low QBER the curves are relatively flat, but beyond a certain QBER threshold each curve exhibits a regime of exponential growth in λ_{OT} . The leakage rate effectively acts as an offset for this threshold: higher leakage allows the protocol to tolerate slightly higher QBER before the bound goes exponential, but at the cost of revealing more information about the raw key.

Even under the the optimistic hardware scenario and a leakage rate closer to the shanon entropy (QBER around 3.3% and assuming a leakage rate of 0.2 is sufficient to achieve near-100% decoding success in our LDPC codes), the minimum required value of λ_{OT} is approximately 4×10^6 . This means that, even the most current optimistic assumptions about the post-processing, several million entangled pairs must be distributed and measured per OT instance to meet the target security level.

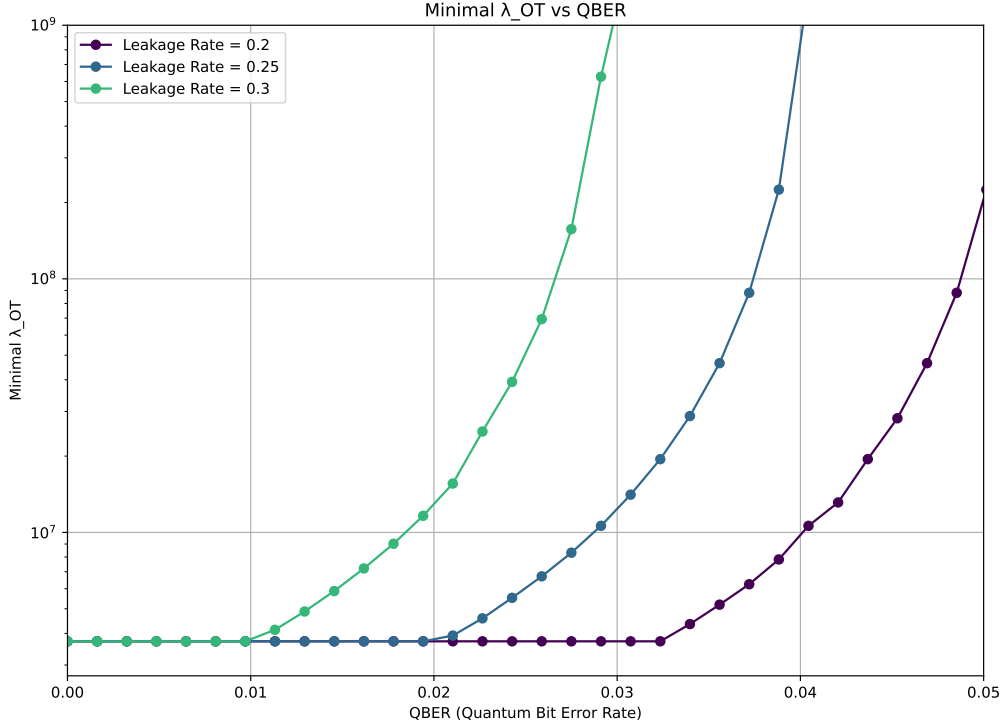


Figure 5.8: Minimal λ_{OT} as a function of the QBER for different leakage rates λ_ℓ .

As discussed in Section 5.2.4, the average time required for a successful OT execution grows approximately linearly with λ_{OT} in our simulator. Combining this with the security bound above implies that the time needed to obtain a secure OT instance also inherits this dependence on QBER: once we enter the point where λ_{OT} grows exponentially in the QBER, the total runtime becomes impractical as well. In particular, transmitting on the order of 4×10^6 states at the per-state transmission rate measured in Section 5.2.4 would take in the order of weeks in our optimistic scenario. This shows that this protocol is not efficient under the optimistic hardware and leakage assumptions.

OT-variants for MPC

As discussed in 3.2, different versions of oblivious transfer are necessary to construct an MPC algorithm. Using the proper reductions, these algorithms can be constructed using another in the following way `1-2 string OT` $\rightarrow$ `1-out-of-N string OT` $\rightarrow$ `Rabin string OT`. Previously, we’ve implemented 1-2 String OT with our QOT protocol. In this chapter we’ll show how we’re able to create 1-out-of-N OT, and use that to create an Rabin-string-OT, which has been used for constructing an MPC algorithm. Furthermore, we note the efficiency of these constructed variants. The code for all constructions can be found in the GitHub repository [19].

6.1 1-2 bit OT

Intuitively, we can just send a single bit with the 1-2 String OT protocol, to gain 1-2 bit OT. However, due to the implementation of our 1-2QOT, this not as straightforward. Because the protocol depends on the matching of random bases and because of the need of bit commitment (Protocol 5.1), multiple bits need to be sent for a successful OT. Two potential solutions for this can be selecting a specific pre-determined bit from the bitstring, or XORing all received values into a single bit. Both variants are implemented as a proof of concept.

Efficiency

As this implementation uses a single instance of the 1-2QOT protocol with very little post-processing, one execution takes essentially the same amount of time as a regular 1-2QOT run.

6.2 1-out-of-N OT

We construct our quantum 1-out-of- N oblivious transfer (1- N OT) from *Protocol 2.1* of Naor and Pinkas [12] by replacing their classical 1-out-of-2 OT subroutine with our 1-out-of-2 quantum OT (1-2QOT). Below is our implementation, with the flowchart in Protocol 6.1.

In our setting, *Alice* is the sender and holds $N = 2^\ell$ messages $X = (X_0, \dots, X_{N-1})$, where each $X_I \in \{0, 1\}^m$. *Bob* is the receiver and holds a choice index $I \in \{0, \dots, N-1\}$. He wishes to learn only X_I , and nothing about the other messages. Alice should not learn which index I Bob chose.

$\text{Bin}(I)$ is the function which gives the binary representation of I . If $\text{Bin}(I) = (i_0, i_1, \dots, i_{\ell-1})$ with each $i_j \in \{0, 1\}$, then i_j denotes the bit of I at position j . We use an AES-based pseudorandom function (PRF) $F_K(\cdot)$, where the key K is a t -bit AES key and the output is truncated to m bits.

Step 1: Setup (offline, by Alice)

In the setup phase, Alice prepares and encrypts all data needed for the OT. This can be performed offline, before Bob chooses his index.

1. Alice samples ℓ independent pairs of random keys (K_j^0, K_j^1) for $j = 0, \dots, \ell - 1$, where each K_j^b is a key for the PRF F_K .
2. For every possible index $I \in \{0, \dots, N - 1\}$, Alice proceeds as follows:
 - (a) Let $\text{Bin}(I) = (i_0, \dots, i_{\ell-1})$.
 - (b) For each position $j \in \{0, \dots, \ell - 1\}$, select the key $K_j^{i_j}$ according to the bit i_j .
 - (c) Compute the mask

$$M_I = \bigoplus_{j=0}^{\ell-1} F_{K_j^{i_j}}(I),$$

where $F_{K_j^{i_j}}(I)$ denotes the PRF evaluated on input I with key $K_j^{i_j}$.

- (d) Using this mask, Alice encrypts each message $Y_I = X_I \oplus M_I$. The values $Y_0, \dots, Y_{N-1}$ can be stored until the online phase.

Step 2: Data exchange (online)

In the online phase, Bob chooses his index I and obtains the keys needed to reconstruct the corresponding mask M_I , without revealing I to Alice.

3. Bob computes the bit representation $\text{Bin}(I) = (i_0, i_1, \dots, i_{\ell-1})$.
For each position $j \in \{0, \dots, \ell - 1\}$, Bob engages in a 1-2QOT instance with Alice, using his choice bit i_j . Which means that for every position:
 - Alice acts as the sender and inputs the pair (K_j^0, K_j^1) .
 - Bob acts as the receiver and, using choice bit i_j , learns only the key $K_j^{i_j}$.
4. After all 1-2QOT instances are completed, Alice sends all ciphertexts $(Y_0, Y_1, \dots, Y_{N-1})$ to Bob over a classical channel.

Step 3: Data reconstruction (by Bob)

6. Bob now knows the index I , the bits $(i_0, \dots, i_{\ell-1})$, and the keys $\{K_j^{i_j}\}_{j=0}^{\ell-1}$ obtained via 1-2QOT. He recomputes the mask

$$M_I = \bigoplus_{j=0}^{\ell-1} F_{K_j^{i_j}}(I).$$

7. Finally, Bob recovers the desired message by unmasking $X_I = Y_I \oplus M_I$. Since Bob never obtained any keys of the form $K_j^{1-i_j}$, he cannot reconstruct the masks for other indices $I' \neq I$ and thus learns nothing about the remaining messages.

Alice		Bob
<i>Input:</i> Messages $(X_0, \dots, X_{N-1})$ with each $X_I \in \{0, 1\}^m$ <i>Common parameter:</i> $N = 2^\ell$		<i>Input:</i> Choice index $I \in \{0, \dots, N-1\}$
Step 1: Setup (offline)		
1 Samples ℓ random key pairs (K_j^0, K_j^1) for $j = 0, \dots, \ell-1$.		
2 For each $I \in \{0, \dots, N-1\}$:		
Computes $\text{Bin}(I) = (i_0, \dots, i_{\ell-1})$.		
For each j , selects $K_j^{i_j}$ and computes $F_{K_j^{i_j}}(I)$.		
Lets $M_I = \bigoplus_{j=0}^{\ell-1} F_{K_j^{i_j}}(I)$ and sets $Y_I = X_I \oplus M_I$.		
Step 2: Data exchange (online)		
7 Transfers the requested values of $K_j^{i_j}$	$\xrightarrow{1\text{-}2\text{QOT}}$	3 Chooses I and computes $\text{Bin}(I) = (i_0, \dots, i_{\ell-1})$. For each $j = 0, \dots, \ell-1$, runs a 1-2QOT with choice bit i_j and obtains $K_j^{i_j}$.
4 Sends all ciphertexts $Y_0, \dots, Y_{N-1}$ over the classical channel.	$\xrightarrow{\text{Classical}}$	4 Receives $Y_0, \dots, Y_{N-1}$.
Step 3: Data reconstruction		
		5 Using $\{K_j^{i_j}\}_{j=0}^{\ell-1}$, computes $M_I = \bigoplus_{j=0}^{\ell-1} F_{K_j^{i_j}}(I)$
		6 And outputs $X_I = Y_I \oplus M_I$.

Protocol 6.1: Quantum 1-out-of- N OT from 1-2QOT.

Efficiency

The number of calls to the 1-2QOT protocol in the 1-out-of- N OT construction depends on the value of N , so the total execution time scales with this value. In our setting, we have $N = 2^\ell$, so only $\ell = \log_2 N$ instances of the 1-2QOT protocol are required, one for each bit of the index I .

In addition, Naor and Pinkas [12] describe a recursive variant that keeps the number of calls to the underlying QOT subroutine at $O(\log N)$ while further improving communication efficiency. This means that converting from 1-2QOT to 1-out-of- N OT adds only a modest overhead compared to running the base 1-2QOT protocol.

6.3 Rabin-string-OT

As introduced in 3.2, Ishai, Sahai, and Prabhakaran [1] present a compiler that transforms MPC protocols into ones with a stronger security guarantee. This compiler requires Rabin-string-OT as a primitive, which they construct from 1-out-of- N string OT. More specifically, they describe how Rabin-string-OT with rational erasure probability p/q (for positive integers $p < q$) can be realised using 1-out-of- N string OT with constant communication overhead. They construct the following protocol:

1. Alice inputs N strings into the 1-out-of- N string OT, of which a random subset of size p is designated as valid transfers. The remaining $N - p$ inputs are arbitrary data.
2. Bob chooses an index uniformly at random and receives the corresponding string via 1-2-string-OT.
3. Alice reveals which indices among the N positions correspond to the subset of p valid transfers.
4. If Bob's chosen index is not among these p indices, the transfer is considered to have failed. Otherwise, Bob has correctly received the message.

By choosing $N = q$ and placing the actual message in exactly p of these positions, the receiver obtains the message with probability p/q , as required for Rabin-string-OT.

Using this construction together with our previously described reduction from 1-out-of- N OT to 1-2QOT, we can instantiate Rabin-string-OT under the assumption of 1-2QOT. This means that we have obtained all OT variants required for their compiler.

Efficiency

The Rabin-string-OT construction is a variant of 1-out-of- N string OT, so its efficiency is determined by the choice of N . The protocol designer can set N based on the desired granularity of the erasure probability: for example, choosing $N = 10$ allows probabilities in steps of 10%, while $N = 100$ allows steps of 1%, and so on.

Discussion

Based on a combination of techniques from the literature, we implemented a quantum oblivious transfer (QOT) protocol and used it as a blueprint for constructing several OT variants. The implementations were benchmarked to assess the feasibility of using these OT variants within MPC. In this discussion, we relate our findings to the previously formulated research questions.

Evaluating 1-2 QOT

Our first research question was: *“Is it possible to implement a quantum-based oblivious transfer protocol inside current entanglement-based quantum-network simulation platforms?”*

To answer this question, we implemented a 1-2 QOT algorithm in an entanglement-based simulator, as described in Chapter 5. For error correction, both Reed–Solomon and low-density parity-check (LDPC) codes were implemented. For hashing, we tested both Toeplitz and Carter–Wegman constructions. Several components and the overall protocol were benchmarked. Where applicable, we used both current and optimistic simulation scenarios for context.

Through testing, we found that the LDPC codes were drastically better at correcting errors at lower leakage rates: in the optimistic scenario, a leakage rate between 0.3 and 0.4 yielded a success-rate of near 100%, whereas Reed–Solomon error correction often required roughly twice that leakage rate for comparable performance, as shown in Figure 5.3. We however suspect that by using more optimized error correction algorithms, the same results can be achieved by using lower leakage rates. For comparison, Hanouz et al. [5] realized possible leakage rates between 0.22 and 0.28 for a QBER of 0.025 by applying alternative error-correction matrices with the help of rate-adaptation techniques.

For hashing, Figure 5.4 shows that the Carter–Wegman hash scaled more efficiently with the input size than the Toeplitz hash. An interesting result, as the Toeplitz hash is the more common used option in the literature. However, the hashing time remained insignificant compared to the time needed to successfully transmit and measure an EPR pair, as shown in Figure 5.5: accounting for only 0.001% to 0.0016% of the runtime.

According to the security bound, we find that, even under the optimistic hardware scenario, and the security parameter set to $\epsilon_{sec} = 2^{-22}$, the minimal value of λ_{OT} needed to be 4×10^6 . The current rate that entangled EPR pairs can be transmitted and measured is low, at 200 states per minute. This means that one OT with this bound could take in the order of weeks to complete.

To summarise, it is possible to implement the required OT protocol inside an entanglement-based simulator. Although the current hardware is inefficient, which makes the performance unworkable for practical uses, it does provide a working foundation for when hardware will improve.

In the future, performance could be improved in both the hardware and post-processing stages. If the cycle time improves, each quantum state can be transmitted more quickly. Another way to reduce the total time is to improve the reliability of the link: according to the security bound, the required number of transmitted states drastically decreases as the fidelity increases, since fewer bits need to be leaked to correct bit flips. This can also be addressed in software by using more effective error-correcting codes, which would further reduce the amount of leaked information.

Evaluating QOT variants

Our second research question was: *“Is it possible to construct the QOT variants needed for a quantum-secure multiparty computation protocol?”*

To answer this question, we first described the different OT protocols needed for constructing a secure multiparty computation protocol in Section 3.2, and built these OT variants using the 1-2 QOT as a base in Chapter 6. We found that 1-2 bit OT, 1-out-of- N OT, and Rabin OT can all be constructed from 1-2 QOT. In practice, this means that our QOT implementation suffices as a black-box realisation of 1-2 string OT.

The performance of these QOT variants is therefore closely tied to the performance of the underlying 1-2 QOT protocol. The 1-2 bit OT construction uses a single invocation of 1-2 QOT and thus has essentially the same runtime as a single 1-2 QOT execution. Our 1-out-of- N OT construction requires $\log_2 N$ calls to the 1-2 QOT protocol, one for each bit of the choice index, which adds a logarithmic overhead in N on top of the base QOT cost. For Rabin-string-OT, the overhead is determined by the chosen parameter N , but for reasonable values of N the additional classical work remains negligible compared to the quantum communication. Since the quantum transfer dominates the runtime and the required security parameters already lead to long execution times in the base protocol, these constructions do not significantly change the overall feasibility, the main bottleneck remains the cost of running 1-2 QOT.

To summarise, it is possible to construct the needed OT variants for a quantum-secure multiparty computation protocol, with performance in the same order of magnitude as the underlying 1-2 QOT.

Future efforts could focus on integrating these OT variants into existing or new MPC protocols. These protocols can then be evaluated with respect to their end-to-end runtime and resource requirements, and further optimisations could be explored to reduce the number of required QOT invocations or improve the efficiency of the underlying implementation.

7.1 Ethical Considerations

For this Informatica Bachelor’s thesis, we are required to reflect on the ethical aspects of our research. The goal of MPC is to support secure computation on data held by multiple parties. Practical applications could be found in collaborative research or in the processing of data within medical institutions. As discussed in Chapter 2, Shor’s algorithm implies that many currently deployed public-key techniques are vulnerable to the increasing computational power of quantum computers. By using QOT, this thesis contributes in an ethical manner to improve the future resilience of digital communication, so that critical applications that depend on secure computation can continue to operate.

Furthermore, during the research for this thesis, no personal or sensitive data was processed.

Conclusion

The main research question of this thesis was: **“Can quantum oblivious transfer serve as an improvement for classical OT within existing multiparty computation protocols?”**

To investigate this, we implemented a 1-out-of-2 quantum oblivious transfer protocol in an entanglement-based quantum-network simulator and extended it to the oblivious transfer variants typically required by secure multiparty computation protocols. The implementation combined quantum communication with classical post-processing steps such as bit commitment, error correction, and hashing, which was evaluated under both current and optimistic hardware assumptions provided by Maiti et al. [17].

The experiments show that the protocol can be implemented in the SquidASM simulated quantum-entanglement environment. Through benchmarking, we found that in this implementation, LDPC codes outperform Reed–Solomon codes at lower leakage rates, and Carter–Wegman hashing scales more efficiently, with relation to the number of bits, than Toeplitz hashing. This post-processing only contributes to a small fraction of the total runtime. The dominant cost is the distribution and measurement of sufficiently many high-fidelity entangled pairs. When the security bound is measured with the observed parameters, the number of required quantum states becomes in the order of millions for an overall security of $\epsilon_{sec} = 2^{-22}$. This makes even with optimistic entanglement generation rates, the resulting QOT protocol too slow for practical use in current settings, as it would take a matter of weeks for one OT.

We also showed that the required OT variants for quantum-secure MPC can all be constructed from our 1-2 QOT implementation. This means that, from a functionality perspective, QOT can serve as a black-box replacement for classical 1-2 string OT in protocols that use these primitives. However, the runtime of all constructed variants remains dominated by the cost of the underlying 1-2 QOT executions, so their overall performance is still constrained by the same practical limitations as the base protocol. Improving the efficiency of 1-2 QOT therefore remains the main challenge for making these OT-based MPC constructions practical.

Main findings

We found experimentally that entanglement based quantum oblivious transfer can realise the same OT functionality as classical schemes and can therefore in principle support existing MPC protocols. However, in terms of concrete performance on current and optimistic quantum-network hardware, getting rid of PKI does not justify the time requirements associated with QOT. The time needed to generate and process enough entangled states to reach the desired security level is significantly higher in practical MPC deployments than both classical OT as well as QOT based on the prepare and measure architecture. As a result, entanglement-based QOT can currently be viewed as a proof-of-concept, but not as a viable replacement.

Future work

Future work should concentrate on closing this performance gap. On the quantum side, this includes improving entanglement generation rates, fidelities, and cycle times in quantum networks. On the classical post-processing side, it involves refining error-correction techniques to reduce leakage and the number of required quantum states. Despite these shortcomings, it remains interesting to integrate QOT into an actual MPC protocol and measure the resulting end-to-end runtime. This would clarify how the overall cost scales with the structure and number of QOT invocations.

Bibliography

- [1] Yuval Ishai, Amit Sahai, and Manoj Prabhakaran. “Founding Cryptography on Oblivious Transfer – Efficiently”. en. In: *Advances in Cryptology – CRYPTO 2008*. Vol. 5157. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 572–591. ISBN: 978-3-540-85173-8 978-3-540-85174-5. DOI: 10.1007/978-3-540-85174-5_32. URL: http://link.springer.com/10.1007/978-3-540-85174-5_32 (visited on 04/09/2026).
- [2] Sushil Jajodia, Pierangela Samarati, and Moti Yung, eds. *Encyclopedia of Cryptography, Security and Privacy*. en. Cham: Springer Nature Switzerland, 2025. ISBN: 978-3-030-71520-5 978-3-030-71522-9. DOI: 10.1007/978-3-030-71522-9. URL: <https://link.springer.com/10.1007/978-3-030-71522-9> (visited on 04/07/2026).
- [3] P.W. Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. en. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. Santa Fe, NM, USA: IEEE Comput. Soc. Press, 1994, pp. 124–134. ISBN: 978-0-8186-6580-6. DOI: 10.1109/SFCS.1994.365700. URL: <http://ieeexplore.ieee.org/document/365700/> (visited on 06/09/2026).
- [4] Yehuda Lindell. “Secure multiparty computation”. en. In: *Communications of the ACM* 64.1 (Jan. 2021), pp. 86–96. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3387108. URL: <https://dl.acm.org/doi/10.1145/3387108> (visited on 04/07/2026).
- [5] Lucas Hanouz et al. *Toward multi-purpose quantum communication networks: from theory to protocol implementation*. en. arXiv:2603.02923 [quant-ph]. Mar. 2026. DOI: 10.48550/arXiv.2603.02923. URL: <http://arxiv.org/abs/2603.02923> (visited on 04/07/2026).
- [6] Claude Crépeau and Joe Kilian. “Achieving oblivious transfer using weakened security assumptions”. en. In: *Advances in Cryptology — CRYPTO’ 88*. Ed. by Shafi Goldwasser. Vol. 403. Series Title: Lecture Notes in Computer Science. New York, NY: Springer New York, 1988, pp. 42–52. ISBN: 978-0-387-97196-4 978-0-387-34799-8. DOI: 10.1007/0-387-34799-2_1. (Visited on 05/13/2026).
- [7] Charles H. Bennett et al. “Experimental quantum cryptography”. en. In: *Journal of Cryptology* 5.1 (Jan. 1992), pp. 3–28. ISSN: 0933-2790, 1432-1378. DOI: 10.1007/BF00191318. URL: <http://link.springer.com/10.1007/BF00191318> (visited on 05/13/2026).
- [8] Hoi-Kwong Lo. “Insecurity of quantum secure computations”. en. In: *Physical Review A* 56.2 (Aug. 1997), pp. 1154–1162. ISSN: 1050-2947, 1094-1622. DOI: 10.1103/PhysRevA.56.1154. URL: <https://link.aps.org/doi/10.1103/PhysRevA.56.1154> (visited on 04/08/2026).
- [9] Ivan Damgaard et al. “Improving the Security of Quantum Protocols via Commit-and-Open”. In: (2009). Version Number: 4. DOI: 10.48550/ARXIV.0902.3918. URL: <https://arxiv.org/abs/0902.3918> (visited on 05/13/2026).
- [10] James Bartusek et al. *One-Way Functions Imply Secure Computation in a Quantum World*. Version Number: 3. 2020. DOI: 10.48550/ARXIV.2011.13486. URL: <https://arxiv.org/abs/2011.13486> (visited on 04/23/2026).

- [11] Eleni Diamanti et al. *A Practical Protocol for Quantum Oblivious Transfer from One-Way Functions (Preprint)*. Version Number: 2. 2026. DOI: 10.48550/ARXIV.2406.09110. URL: <https://arxiv.org/abs/2406.09110> (visited on 04/23/2026).
- [12] Moni Naor and Benny Pinkas. “Computationally Secure Oblivious Transfer”. en. In: *Journal of Cryptology* 18.1 (Jan. 2005), pp. 1–35. ISSN: 0933-2790, 1432-1378. DOI: 10.1007/s00145-004-0102-6. URL: <http://link.springer.com/10.1007/s00145-004-0102-6> (visited on 06/14/2026).
- [13] Michael O Rabin. “How to Exchange Secrets with Oblivious Transfer”. In: (1981).
- [14] Axel Dahlberg et al. “NetQASM – A low-level instruction set architecture for hybrid quantum-classical programs in a quantum internet”. In: (2021). Version Number: 2. DOI: 10.48550/ARXIV.2111.09823. URL: <https://arxiv.org/abs/2111.09823> (visited on 04/08/2026).
- [15] Tim Coopmans et al. “NetSquid, a NETWORK Simulator for QUantum Information using Discrete events”. en. In: *Communications Physics* 4.1 (July 2021), p. 164. ISSN: 2399-3650. DOI: 10.1038/s42005-021-00647-8. URL: <https://www.nature.com/articles/s42005-021-00647-8> (visited on 05/12/2026).
- [16] *SquidASM Documentation*. Documentation. URL: <https://squidasm.readthedocs.io/en/latest/>.
- [17] Soubhadra Maiti et al. *Requirements for Teleportation in an Intercity Quantum Network*. en. arXiv:2602.04869 [quant-ph]. Feb. 2026. DOI: 10.48550/arXiv.2602.04869. URL: <http://arxiv.org/abs/2602.04869> (visited on 05/20/2026).
- [18] M. Zwerger et al. “Quantum repeaters based on trapped ions with decoherence free subspace encoding”. In: (2016). Version Number: 2. DOI: 10.48550/ARXIV.1611.07779. URL: <https://arxiv.org/abs/1611.07779> (visited on 06/10/2026).
- [19] D.R.B. van Wijk. *1-2QOT*. URL: <https://github.com/vanwijkdave/1-2QOT>.
- [20] Pedro Cosme. “A Step-by-Step Introduction to Error-Correcting Codes: From Hamming to Low-Density Parity-Check (LDPC)”. en. In: (2026). DOI: 10.13140/RG.2.2.31955.54563. URL: <https://www.researchgate.net/doi/10.13140/RG.2.2.31955.54563> (visited on 06/03/2026).
- [21] Joschka Roffe. *LDPC: Python tools for low density parity check codes*. 2022. URL: <https://pypi.org/project/ldpc/>.
- [22] Adomas Baliuka and Elsa Dupraz. *LDPC4QKD: LDPC Codes for Rate Adaptive Distributed Source Coding*. Oct. 2021. DOI: 10.5281/ZENODO.5589543. URL: <https://zenodo.org/record/5589543> (visited on 06/09/2026).
- [23] Brown Andrew. *unireedsolomon*. 2023. URL: <https://pypi.org/project/unireedsolomon/>.

1-2 QOT Protocol

Alice		Bob
<i>Input:</i> Messages (m_0, m_1)		<i>Input:</i> Bit $b \in \{0, 1\}$
<i>Common input:</i> $\lambda_{OT}, \lambda_{PQS}, h_i(), PRG(), Comm()$		
State Distribution		
1.a Chooses $\theta_A \xleftarrow{\$} \{+, \times\}^{2\lambda_{OT}}$, and prepares EPR pair (2 qubits $ x_{AB}\rangle$), of which she measures $ x_A^{\theta_A}\rangle$ and sends bob x_B .	Entangled States $\rightarrow$	1.b Measures his qubit with random basis $\hat{\theta} \xleftarrow{\$} \{+, \times\}^{2\lambda_{OT}}$ to obtain $\hat{x} \in \{0, 1\}^{2\lambda_{OT}}$.
Bit Commitment		
2. Receives commitment t .	$\xleftarrow{\text{Classical}}$	2. Sends public commitment $t = Comm(\hat{x}) + Comm(\hat{\theta})$.
3. Chooses random subset $T \subset [2\lambda_{OT}]$ with $ T = \lambda_{OT}$ and sends T .	$\xrightarrow{\text{Classical}}$	3. Receives T .
4. Checks $1 - \text{Verif}(t, d)/\lambda_{OT} \leq Q_{\text{tol}}$, otherwise aborts.	$\xleftarrow{\text{Classical}}$	4. Opens and sends $d = \text{Open_comm}(\hat{x}_{ T}) + \text{Open_Comm}(\hat{\theta}_{ T})$.
Basis Reconciliation		
5. Sends θ_A .	$\xrightarrow{\text{Classical}}$	5. Receives θ_A .
6. Receives I_0 and I_1 .	$\xleftarrow{\text{Classical}}$	6. Computes $I_b = \{i \notin T : \theta_{A_i} = \hat{\theta}_i\}$ and $I_{\bar{b}} = \{i \notin T : \theta_{A_i} \neq \hat{\theta}_i\}$, sends I_0, I_1 .
7. Determines $x_0 = \{x_i : i \in I_0\}$ and $x_1 = \{x_i : i \in I_1\}$.		
Error Correction		
8. Generates and sends $\sigma_0 = \text{Synd}(x_0)$, $\sigma_1 = \text{Synd}(x_1)$,	$\xrightarrow{\text{Classical}}$	8. Receives σ_0, σ_1 . Corrects $\hat{x}_b = \text{Correct}(\hat{x}_{ I_b}, \sigma_b)$.
Encryption		
9. Generates and sends seeds $s_0, s_1 \xleftarrow{\$} \{0, 1\}^{n_0 + \lambda_{PQS}}$. Constructs and sends $E_0 = PRG(h_0(x_0, s_0)) \oplus m_0$, $E_1 = PRG(h_1(x_1, s_1)) \oplus m_1$.	$\xrightarrow{\text{Classical}}$	9. Receives s_0, s_1, E_0, E_1 Decrypts $m_b = PRG(h_b(\hat{x}_b, s_b)) \oplus E_b$.

Simulating Parameters

B.1 Current

```

1 qdevice_cfg: &qdevice_cfg
2   num_qubits: 2
3
4   T1: 62_000_000 # 62 ms
5   T2: 62_000_000 # 62 ms
6
7   init_time: 65_000 # 65 us
8   single_qubit_gate_time: 12_000 # 12 us
9   two_qubit_gate_time: 107_000 # 107 us
10  measure_time: 2_000_000 # 2 ms
11
12  single_qubit_gate_depolar_prob: 0.02 # fid=0.99
13  two_qubit_gate_depolar_prob: 0.067 # fid=0.95
14
15  qlink_cfg: &qlink_cfg:
16    fidelity: 0.88
17    t_cycle: 425_000
18    prob_success: 5.95e-5 # d=50 km
19
20
21  stacks:
22    - name: Alice
23      qdevice_typ: generic
24      qdevice_cfg:
25        <<: *qdevice_cfg
26    - name: Bob
27      qdevice_typ: generic
28      qdevice_cfg:
29        <<: *qdevice_cfg
30
31  links:
32    - stack1: Alice
33      stack2: Bob
34      typ: depolarise
35      cfg:
36        <<: *qlink_cfg
37
38  clinks:
39    - stack1: Alice
40      stack2: Bob
41      typ: instant

```

B.2 Optimistic

```
1 qdevice_cfg: &qdevice_cfg
2   num_qubits: 2
3
4   T1: 4_000_000_000 # 4 s
5   T2: 4_000_000_000 # 4 s
6
7   init_time: 65_000 # 65 us
8   single_qubit_gate_time: 15_000 # 15 us
9   two_qubit_gate_time: 200_000 # 200 us
10  measure_time: 2_000_000 # 2 ms
11
12  single_qubit_gate_depolar_prob: 0.004 # fid=0.998
13  two_qubit_gate_depolar_prob: 0.004 # fid=0.997
14
15 qlink_cfg: &qlink_cfg:
16   fidelity: 0.95
17   t_cycle: 425_000 # 425 us, d=50 km
18   prob_success: 1.43e-3 # d=50 km
19
20
21 stacks:
22 - name: Alice
23   qdevice_typ: generic
24   qdevice_cfg:
25     <<: *qdevice_cfg
26 - name: Bob
27   qdevice_typ: generic
28   qdevice_cfg:
29     <<: *qdevice_cfg
30
31 links:
32 - stack1: Alice
33   stack2: Bob
34   typ: depolarise
35   cfg:
36     <<: *qlink_cfg
37
38 clinks:
39 - stack1: Alice
40   stack2: Bob
41   typ: instant
```

Listing B.2: Configuration used for simulating the optimistic scenario